the committed JSON manifest from tamland-dedicated and use this as input to execute Tamland.

Capacity planning reports and Capacity Warnings

Based on Tamland's forecasting data, we generate reports to display forecasting information and enable teams to act on capacity warnings by creating capacity warnings in a GitLab issue tracker.

The Scalability::Observability team maintains an internal GitLab project called gitlab-dedicated.
This project contains a scheduled CI pipeline to regularly produce a static site deployed to GitLab Pages (only available internally).
It also contains functionality to create and manage capacity warnings in the issue tracker of this project.

CI configuration for this project contains a list of tenants along with their respective metadata (e.g. AWS account, codename, etc.).

For each configured tenant, the CI pipeline uses a central IAM role in the amp account.
With this role, a tenant-specific IAM role can be assumed, which has read-only access to the respective S3 bucket containing the tenant's forecasting data.

The CI pipeline produces a standard Tamland report for each tenant and integrates all individual reports into a single static site.
This site provides unified access to capacity forecasting insights across tenant environments.

Along with the report, the CI pipeline also reacts to predicted SLO violations and creates a capacity warning issue in the project's issue tracker.
As the tracker is being used for all GitLab Dedicated tenants, we employ a ~tenant:CN label to distinguish tenant environments (e.g. we use ~tenant:C1 for the tenant with codename C1).
These issues contain further information about the tenant and component affected, along with forecasts and status information.
The intention here is to create visibility into predicted SLO violations and provide a way for the Dedicated team to engage with capacity warnings directly (e.g. for discussion, work scheduling etc.).

Overall, the Dedicated teams and operators use the Tamland report and issue tracker to act on capacity warnings.

In order to get started, we suggest the Dedicated group to take a regular pass across the capacity warnings and triage those.
For additional visibility, we may want to consider enabling getting Slack updates sent out for new capacity warnings created.

Alternative Solution

Tamland as a Service (not chosen)

An alternative design, we don't consider an option at this point, is to set up Tamland as a Service and run it fully outside of tenant environments.

In this design, a central Prometheus/Mimir instance is needed to provide the metrics data for Tamland.
Dedicated tenants use remote-write to push their Prometheus data to the central Mimir instance.

Tamland is set up to run on a regular basis and consume metrics data from the single Mimir instance.
It stores its results and cache in S3, similar to the other design.

In order to execute forecasts regularly, we need to provide an execution environment to run Tamland in.
With an increasing number of tenants, we'd need to scale up resources for this cluster.

This design has not been chosen because of both technical and organisational concerns:

1. Our central Mimir instance currently doesn't have metrics data for Dedicated tenants as of the start of FY24Q3.
1. Extra work required to set up scalable execution environment.
1. Mimir is considered a bottleneck as it provides data for all tenants and this poses a risk of overloading it when we execute the forecasting for a high number of tenants.
1. We strive to build out Tamland into a tool of more general use. We expect a better outcome in terms of design, documentation and process efficiency by building it as a tool for other teams to use and not offering it as a service. In the long run, we might be able to integrate Tamland (as a tool) inside self-managed environments or publish Tamland as an open source forecasting tool. This would not be feasible if we were hosting it as a service.

SECTION: engineering/architecture/design-documents/cdot_error_reporting/_index.md

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

CustomersDot is a powerful platform designed for customers to seamlessly purchase and manage their subscriptions. For SaaS applications, CustomersDot communicates with GitLab.com through API calls to update subscription details in real-time. For self-managed (SM) applications and dedicated instances, it generates a license for installation on the customer's system. The latest version enhances this process by enabling license installation through the cloud, offering greater convenience and flexibility.

CustomersDot integrates with several third-party tools, such as Zuora and Salesforce. Zuora is an end-to-end order-to-revenue SaaS platform that drives GitLab's quoting, billing, revenue collection, revenue recognition, and subscription metrics reporting. It also supports the back-office teams responsible for these operations.

Motivation

From a SOX compliance perspective, it is crucial to identify and track any errors that occur during the purchase, update and provisioning of subscriptions within the CustomersDot application. Currently, we utilize the Provision Tracking System to monitor the provisioning process after a subscription is created or updated. However, there is a tracking gap for certain revenue recognition-impacting errors that occur before a subscription purchase or update. It is crucial to address this gap to ensure comprehensive error tracking and improve the accuracy of revenue recognition processes.

Goals

The goal is to deliver a tracking and monitoring system for revenue-impacting errors that occur during the purchase, update, renewal, and reconciliation flows. This will cover the following aspects of CustomersDot integrations (Zuora, Salesforce for Q3 & others in Q4 2025).

A spike issue for the proposal can be found at gitlab-org/customers-gitlab-com10430.

Enhanced visibility for stakeholders (Accessing different systems to verify data)
Document edge cases
Automated error monitoring/Alerting system
Document fixes or processes for 3rd party errors (like Zuora unavailability etc)

Proposal

The Fulfillment Platform group currently handles the manual process of monitoring the described errors. Each week, an engineer is assigned to a Job monitoring issue, where they must gather a collection of errors daily by running a script. These errors are then reviewed individually to identify actionable items, such as those not related to user validation, payment method validation, or location issues. The engineer manually resolves these errors with the use of console or Admin panel.

To improve efficiency and results, we need to automate this process.

Develop a new system for error tracking to improve the monitoring of provisioning and revenue recognition errors.
  Create a new database table to store error logs.
  Add a new view in the admin panel to display and manage these errors.
Enhance the codebase to automatically store relevant errors in the new database.
Implement Slack notifications to alert the team whenever a new error is encountered.
Automate issue creation:
  Generate a daily issue summarizing new errors encountered each day.
  Generate a weekly issue summarizing all errors encountered throughout the week.
Set up a cron job to run daily as an audit, tracking reconciliations and renewals that failed to process, and add them to the error log.

Design and implementation details

DB schema

```
mermaid
erDiagram
  "ErrorMonitorings" {
    integer id PK
    type string "null:false"
    text message "null:false"
    string code
    string errortype "limit: 1000"
    string status "null:false"
    string gitlabissueiid
    string detailedgitlabissueiid
    text backtrace
    jsonb payload
    text notes
    datetime createdat "null:false"
    datetime updatedat "null:false"
  }
```

The errormonitorings table is designed to store meaningful errors that are valuable for monitoring as they occur. Most of the columns are self explanatory. Taking an example of the current logging in google cloud.

code -> VALIDATIONERROR
errortype -> This code is not valid. Try re-entering the code from your email.
message -> Subscription update failed

The errormonitorings table uses Single Table Inheritance (STI) to store different types of errors. In Rails, the type column enables this STI pattern, allowing multiple error types to be stored in the same table.

Currently, we are using two types: 'RevenueImpact' and 'SalesforceErrors', both of which inherit from the base ErrorMonitoring model.

We are tagging error messages with fulfillmentjobmonitoring to store the Revenue Impact errors within the codebase and using GCloud to look up and resolve them individually.

We are tagging Salesforce error messages with salesforceerrormonitoring to store the errors related to Salesforce within the codebase.

Errors will continue to be addressed individually. As soon as an error is encountered, we send an immediate notification to the designated Slack channel, through the background job, to ensure timely resolution.

Error states

The status column stores the state of the error encountered.

```
mermaid
flowchart TD
```

```
    A[Error encounterd] -->|Save it in database| B(State - Needs Attention)
    B --> |Nothing needs to be done| D[Ignored]
    B --> |Error being looked into| F[State - In Progress]
    F --> C[Create/Link GitLab Issue, optional]
    F --> |Issue Closed| G[State - Resolved]
```

Workflow

```
mermaid
sequenceDiagram
  autonumber
  actor C as Customer
  participant CD as CustomersDot
  participant Z as Zuora
  participant ER as Error reporting System
  participant S as Slack
  participant G as GitLab
  actor E as Engineer

  C->>CD: Initiates subscription purchase/update
  CD->>Z: API callback for subscription purchase/update
  Z-->>CD: Returns error
  CD-->>C: Indicates system error
  CD->>ER: Creates errormonitoring record within database
  ER->>S: Feeds Slack notification
  E->>CD: Visits admin page & debug the error
  E->>G: Creates GitLab issue (optional)
  E->>ER: Changes Status of errormonitoring record
  par
    E->>G: Triage issue and status update
    G->>ER: Auto-updates errormonitoring record on resolution
  end
```

SECTION: engineering/architecture/design-documents/cdot_namespace_provision_sync/_index.md

{{< engineering/design-document-header >}}

Summary

As a part of aligning provisioning between Self-Managed/Dedicated and GitLab.com, we are restructuring the way provision for namespace works on GitLab.com.

Motivation

The work for this will align the provisioning for GitLab.com closer to the way Self-Managed/Dedicated is provisioned.

For GitLab.com, we will record the params generated for the Namespace provisioning on a new gitlabnamespacessyncs table, along with each attempt for a sync and the result status via a gitlabnamespacesssyncattempts table. This is similar to the way we handle Licenses for Self-Managed and results in bringing both provisioning processes closer.

Goals

The goal of this blueprint is to produce:

- an architectural design on how the namespace will be provisioned via the new process
- an iteration plan to achieve the chosen design

Proposal

We want to create a new table gitlabnamespacesyncs that will hold the records for the generated namespace provision params. A gitlabnamespacessync record will have many gitlabnamespacessyncattempts that will log the status of gitlabnamespacessync. The states can be [started, failed, skipped, completed].

Whenever a gitlabnamespacessync record is created, it will always have an associated gitlabnamespacessyncattempt record with started state. We will then make an internal HTTP request to GitLab to provision the namespace with the associated params. The params will have provision information for all the resources to be provisioned: baseproduct, computeminutes, storage, addonpurchases]. See the [API Contract for more information. During the provisioning to GitLab, provision to other resources is continued even if any of the resources provisioning fails. For instance, if the Compute Minutes resource provisioning fails, provisioning for the Storage and AddOnPurchase resources is continued.

Based on the response of the provision sync, we will update the state of the gitlabnamespacessyncattempt record. The status will be updated to completed for a 200 OK response, and failed for any other.

Based on the failed response code we will perform further action:

- 5XX : This is a Server error and the sync will be retried a few times
- 4XX : This is a validation error and will be related to the params generation by the Client. We will log it and investigate on case by case basis.

Iteration 1

For Iteration 1, following are the Sequence Diagram, Flow Chart, Database Table and Internal API we plan to implement on CustomersDot and GitLab.

CustomersDot (CDot) Side

Sequence Diagram

```
mermaid
sequenceDiagram
    participant User
```

```
    participant CDot
    participant GitLab

    alt New subscription or changed info
        User->>CDot: Create/Update subscription
        CDot->>CDot: Create new gitlabnamespacessyncs and<br/> create new
gitlabnamespacessyncattempts (state set to started)
        CDot->>GitLab: Trigger sync
        GitLab-->>CDot: Response
        alt success: 200
            CDot->>CDot: Update state to completed in gitlabnamespacessyncattempts
        else error
            CDot->>CDot: Update state to failed in gitlabnamespacessyncattempts and<br/> handle
response error
        end
    end

    User->>CDot: Manual re-sync request
    CDot->>CDot: Create new gitlabnamespacessyncattempts (state set to started)<br/> for
existing gitlabnamespacessyncs
    CDot->>GitLab: Trigger sync
    GitLab-->>CDot: Response
    alt success: 200
        CDot->>CDot: Update state to completed in gitlabnamespacessyncattempts
    else error
        CDot->>CDot: Update state to failed in gitlabnamespacessyncattempts and<br/> handle
response error
    end
```

Flow chart

```
mermaid
graph TD
    A[Start] --> B{New subscription or changed info?}
    B -->|Yes| C[Create new gitlabnamespacessyncs and<br/> new gitlabnamespacessyncattempts]
    B -->|No| D[No new record created]
    C --> E[Trigger sync to GitLab]
    E --> F[GitLab: response]
    F --> G{success: 200?}
    G -->|Yes| H[Update state in gitlabnamespacessyncattempts]
    G -->|No| I[Update state in gitlabnamespacessyncattempts and handle error response]
    H --> J[End]
    I --> J
    D --> J

    K[Manual re-sync request] --> L[Trigger sync to GitLab]
    L --> M[GitLab: response]
    M --> N{success: 200?}
    N -->|Yes| O[Update state in gitlabnamespacessyncattempts]
```

```
    N -->|No| P[Update state in gitlabnamespacessyncattempts and handle error response]
    O --> Q[End]
    P --> Q
```

Database

```
mermaid
erDiagram
    gitlabnamespacessyncattempts {
        bigint id PK
        bigint namespacessyncid FK
        timestamptz createdat
        timestamptz updatedat
        integer state
    }
    gitlabnamespacessyncs {
        bigint id PK
        timestamptz createdat
        timestamptz updatedat
        integer namespaceid
        string attrssha256
        jsonb attrs
    }

  gitlabnamespacessyncs ||--o{ gitlabnamespacessyncattempts : "has many"
```

GitLab.com side

Sequence diagram

```
mermaid
sequenceDiagram
    participant BW as CDot Background Worker
    participant DB as CDot Database
    participant GL as GitLab
    participant GLDB as GitLab Database

    Note over BW: Namespace provisioning<br>job starts

    BW->>DB: Fetch latest namespacesyncs for namespaceid
    activate DB
    DB-->>BW: Return namespacesync record
    deactivate DB

    Note over BW: Process namespace<br>attributes

    BW->>GL: HTTP POST /api/v4/internal/gitlabsubscriptions/namespaces/:id/provision
    activate GL
```

```
    Note over GL: Step 1:<br>Update gitlabsubscription
    GL->>GLDB: Update gitlabsubscriptions table with plan details
    activate GLDB
    GLDB-->>GL: Confirm subscription update
    deactivate GLDB

    Note over GL: Step 2:<br>Update namespace for:<br>computeminutes & storage
    GL->>GLDB: Update namespaces table with computeminutes & storage limits
    activate GLDB
    GLDB-->>GL: Confirm namespace update
    deactivate GLDB

    Note over GL: Step 3:<br>Provision addonpurchase
    GL->>GLDB: Upsert addonpurchases table
    activate GLDB
    GLDB-->>GL: Confirm add-on purchase upserted
    deactivate GLDB

    Note over GL: Check for any failures
    alt Any step failed
        GL-->>BW: Return error response (422) with failure details
    else All steps succeeded
        GL-->>BW: Return success response (200)
    end
    deactivate GL

    alt Success Response (200)
        Note over BW: Job completed successfully
    else Client Error (4XX)
        Note over BW: Log error details<br>Update gitlabnamespacessync<br>as partially failed
    else Server Error (5XX)
        Note over BW: Schedule job retry<br>Update job status<br>for retry
    end
```

API Contract

A new internal endpoint will be created on GitLab that will do full provision of the namespace.

POST /api/v4/internal/gitlabsubscriptions/namespaces/:id/provision

The endpoint will accept following JSON body structure:

json

```
{
  "provision": {
    "baseproduct": {
      "plancode": "stringvalue",
      "startdate": "2023-06-01",
      "enddate": "2024-05-31",
```

```
  "seats": 100,
  "maxseatsused": 90,
  "trial": false,
  "trialstartson": "2023-07-01",
  "trialendson": "2024-07-01",
  "autorenew": true
},
"computeminutes": {
  "sharedrunnersminuteslimit": 50000,
  "extrasharedrunnersminuteslimit": 10000,
  "packs": [
    {
      "purchasexid": "purchase1",
      "numberofminutes": 5000,
      "expiresat": "2023-12-31"
    },
    {
      "purchasexid": "purchase2",
      "numberofminutes": 5000,
      "expiresat": "2024-06-30"
    }
  ]
},
"storage": {
  "additionalpurchasedstoragesize": 1000,
  "additionalpurchasedstorageendson": "2024-05-31"
},
"addonpurchases": {
  "duopro": [
    {
      "quantity": 100,
      "startedon": "2023-06-01",
      "expireson": "2024-05-31",
      "purchasexid": "purchase123",
      "trial": false
    }
  ],
  "duoenterprise": [
    {
      "quantity": 100,
      "startedon": "2023-06-01",
      "expireson": "2024-05-31",
      "purchasexid": "purchase123",
      "trial": false
    }
  ],
  "productanalytics": [
    {
      "quantity": 100,
      "startedon": "2023-06-01",
```

```
          "expireson": "2024-05-31",
          "purchasexid": "purchase123",
          "trial": false
        }
      ]
    }
  }
}
```

Response

1. 200 : Successful Request
1. 400 : Bad Request
1. 401 : Unauthorized Request
1. 404 : Namespace not found
1. 422 : Unprocesssable Entity
1. 500: Server Error

SECTION: engineering/architecture/design-documents/cdot_orders/_index.md

{{< engineering/design-document-header >}}

Summary

The GitLab Customers Portal is an application separate from the GitLab product that allows GitLab Customers to manage their account and subscriptions, tasks like purchasing additional seats. More information about the Customers Portal can be found in the GitLab docs. Internally, the application is known as CustomersDot (also known as CDot).

GitLab uses Zuora's platform to manage their subscription-based services. CustomersDot integrates directly with Zuora Billing and treats Zuora Billing as the single source of truth for subscription data.

CustomersDot stores some subscription and order data locally, in the form of the orders database table, which at times can be out of sync with Zuora Billing. The main objective for this blueprint is to lay out a plan for improving the integration with Zuora Billing, making it more reliable, accurate, and performant.

Motivation

Working with the Order model in CustomersDot has been a challenge for Fulfillment engineers. It is difficult to trust Order data as it can get out of sync with the single source of truth for subscription data, Zuora Billing. This has led to bugs, confusion and delays in feature development. An epic exists for aligning CustomersDot Orders with Zuora objects which lists a variety of issues related to these data integrity problems. The motivation of this blueprint is to develop a better data architecture in CustomersDot for Subscriptions and associated data models which builds trust and reduces bugs.

Goals

This re-architecture project has several multifaceted objectives.

- Increase the accuracy of CustomersDot data pertaining to Subscriptions and its entitlements. This data is stored as Order records in CustomersDot - it is not granular enough to represent what the customer has purchased, and it is error prone as shown by the following issues:
  - Multiple order records for the same subscription
  - Multiple subscriptions active for the same namespace
  - Support Multiple Active Orders on a Namespace
- Continue to align with Zuora Billing being the SSoT for Subscription and Order data.
- Decrease dependency and reliance on Zuora Billing uptime.
- Improve CustomersDot performance by storing relevant Subscription data locally and keeping it in sync with Zuora Billing. This could be a key piece to making Seat Link more efficient and reliable.
- Eliminate confusion between CustomersDot Orders, which contain data more closely resembling a Subscription, and Zuora Orders, which represent a transaction between a customer and merchant and can apply to multiple Subscriptions.
  - The CustomersDot orders table contains a mixture of Zuora Subscription and trials, along with GitLab-specific metadata like sync timestamps with GitLab.com. GitLab does not store trial subscriptions in Zuora at this time.

Proposal

As the list of goals above shows, there are a good number of desired outcomes we would like to see at the end of implementation. To reach these goals, we will break this work up into smaller iterations.

1. Phase one: Build models for Zuora subscriptions local copy

    The first iteration focuses on creating the foundation for the local copy for Zuora Subscription objects, including Rate Plans, Rate Plan Charges, and Rate Plan Charge Tiers, in CustomersDot. This involves creating the database tables and models for the local copy of resources.

    Phase 1: Build Zuora Cache Models (&11751)

1. Phase two: Implement sync and backfill of Zuora subscriptions local copy

    The second iteration involves establishing a sync between Zuora and the newly introduced models. Additionally, existing Zuora Subscription data will need to be backfilled to ensure seamless integration and data consistency.

    Phase 2: Implement Zuora Cache Sync and Backfill (&13630)

1. Phase three: Utilize Zuora subscriptions local copy

    In the third phase, the objective is to leverage the Zuora subscriptions local copy introduced in phase one and synchronized in phase two. The focus will be on replacing any code in CustomersDot that currently makes read requests to Zuora for Subscription data with

ActiveRecord queries. This shift should lead to a significant performance improvement.

Phase 3: Utilize Zuora Cache Models (&11752)

1. Phase four: Transition from Order to Subscription

The next iteration focuses on trimming down the Order model and resolving data consistency issues.

Phase 4: Replace CDot Order with Subscription (&11753)

- Note: The implementation of the local models doesn't match a traditional cache, as such it was decided to refer to them as local copy instead. The references to cache were updated accordingly, except the names of the completed and in progress issues.

Design and implementation details

Phase one: Build models for Zuora subscriptions local copy

The first phase for this blueprint focuses on adding new models for caching Zuora Subscription data locally in CustomersDot. These local data models will allow CustomersDot to query the local database for Zuora Subscriptions. Currently, this requires querying directly to Zuora which can be problematic if Zuora is experiencing downtime. Zuora also has rate limits for API usage which we want to avoid as CustomersDot continues to scale.

This phase will consist of writing migrations to create the new database tables and building the new data models. This involves analyzing what data attributes from these Zuora resources are needed by the CustomersDot application and ensuring they are built into the migrations with appropriate data types and limitations. The new data models will need validations and assocations set up as well.

Proposed DB schema

```
mermaid
erDiagram
  "Zuora::Local::Subscription" ||--|{ "Zuora::Local::RatePlan" : "has many"
  "Zuora::Local::RatePlan" ||--|{ "Zuora::Local::RatePlanCharge" : "has many"
  "Zuora::Local::RatePlanCharge" ||--|{ "Zuora::Local::RatePlanChargeTier" : "has many"

  "Zuora::Local::Subscription" {
    string(64) zuoraid PK "id field on Zuora Subscription"
    integer version "null:false"
    integer currentterm
    integer initialterm
    integer renewalterm
    datetime createddate "null:false"
    datetime updateddate "null:false"
    date termstartdate "null:false"
    date termenddate
    date cancelleddate
```

```
  date subscriptionstartdate "null:false"
  date subscriptionenddate
  boolean autorenew "null:false default:false"
  string(3) eoastarterbronzeofferacceptedc
  string(3) contractautorenewc
  string(3) turnonautorenewc
  string(3) turnonoperationalmetricsc
  string(3) turnonseatreconciliationc
  string(5) currenttermperiodtype
  string(7) turnoncloudlicensingc
  string(7) marketplaceoffertypec
  string(18) status
  string(64) accountid "null:false"
  string(64) previoussubscriptionid
  string(64) invoiceownerid "null:false"
  string(64) originalid "null:false"
  string(64) rampid
  string(64) createdbyid "null:false"
  string(64) updatedbyid "null:false"
  string(255) name "null:false"
  string(255) opportunityidc
  string(255) renewalsubscriptioncc
  string(255) externalsubscriptionidc
  string(255) externalsubscriptionsourcec
  string(255) gitlabnamespaceidc
  string(255) gitlabnamespacenamec
  string(255) marketplaceagreementidc
  string(255) marketplaceofferidc
  string(255) marketplacefeepercentagec
  string(500) notes
  datetime createdat
  datetime updatedat
}

"Zuora::Local::RatePlan" {
  string(64) zuoraid PK "id field on Zuora RatePlan"
  datetime createddate "null:false"
  datetime updateddate "null:false"
  datetime createdat "null:false"
  datetime updatedat "null:false"
  string(64) subscriptionid FK "null:false"
  string(64) productrateplanid "null:false"
  string(64) createdbyid "null:false"
  string(64) updatedbyid "null:false"
  string(255) name "null:false"
}

"Zuora::Local::RatePlanCharge" {
  string(64) zuoraid PK "id field on Zuora RatePlanCharge"
  integer version "null:false"
```

```
  integer segment "null:false"
  integer quantity
  integer mrr
  integer tcv
  integer dmrc
  integer dtcv
  datetime createddate "null:false"
  datetime updateddate "null:false"
  datetime createdat "null:false"
  datetime updatedat "null:false"
  date effectivestartdate
  date effectiveenddate
  boolean islastsegment "null:false default:false"
  string(9) chargetype
  string(30) pricechangeoption
  string(50) chargenumber "null:false"
  string(64) rateplanid FK "null:false"
  string(64) subscriptionid "null:false"
  string(64) subscriptionownerid "null:false"
  string(64) productrateplanchargeid "null:false"
  string(64) createdbyid "null:false"
  string(64) updatedbyid "null:false"
  string(100) name
  string(500) description
}

"Zuora::Local::RatePlanChargeTier" {
  string(64) zuoraid PK "id field on Zuora RatePlanChargeTier"
  integer tier
  decimal price "precision:18 scale:2"
  datetime createddate "null:false"
  datetime updateddate "null:false"
  datetime createdat "null:false"
  datetime updatedat "null:false"
  string(3) currency
  string(16) priceformat
  string(64) rateplanchargeid FK "null:false"
  string(64) createdbyid "null:false"
  string(64) updatedbyid "null:false"
}
```

Notes

- The namespace Zuora is already taken by the classes used to extend IronBank resource classes. These classes will be moved to the namespace Zuora::Remote to indicate these are intended to reach out to Zuora. This frees up the Zuora namespace to be used for other purposes in later Phases.
- The new models related to Zuora subscriptions local copy will be added to the namespace Zuora::Local. This has nice symmetry with Zuora::Remote and makes it clear which classes refer

to the remote Zuora data source or the local data source.
- All versions of Zuora Subscriptions will be stored in this table to be able to support display of current as well as future purchases when Zuora is down. One of the guiding principles from the Architecture Review meeting on 2023-08-06 was "Customers should be able to view and access what they purchased even if Zuora is down". Given that customers can make future-dated purchases, CustomersDot needs to store current and future versions of Subscriptions.
- zuoraid would be the primary key given we want to avoid the field name id which is magical in ActiveRecord.
- The timezone for Zuora Billing is configured as Pacific Time. Let's account for this timezone as we sync data from Zuora into CDot's cached models to allow for more accurate comparisons.

Phase two: Implement sync and backfill of Zuora subscriptions local copy

The second phase for this blueprint focuses building the mechanisms to keep the local data in sync with Zuora and backfilling the existing data. Ideally, the local cache models would be read-only for most of the application to ensure the data stays in sync. Only the syncing mechanism would have the ability to write to these models.

Keeping data in sync with Zuora

CDot currently receives and processes Order Processed Zuora callouts for Order actions like Update Product (full list). These callouts help to keep CustomersDot in sync with Zuora and trigger provisioning events. These callouts will be important to keeping Zuora::Local::Subscription and related local models in sync with changes in Zuora.

This existing callout would not be sufficient to cover all changes to a Zuora Subscription though. In particular, changes to custom fields may not be captured by these existing callouts. We will need to create custom events and callouts for any custom field in the Zuora subscriptions local copy in CustomersDot for any of these resources to ensure CDot is in sync with Zuora. This should only affect Zuora::Local::Subscription though as no custom fields are used by CustomersDot on any of the other proposed local resources at this time.

Read only models

Given the data stored in these new models are a copy of Zuora data, it will important to ensure these models are modified within the appropriate context, not throughout the application. We want a clear separation when a local copy of a resource can be in "write" mode versus "read-only" mode. This separation helps avoid writing to the local copy of a resource inappropriately or mistakenly. We considered different options as part of this Spike issue.

We aligned on creating a concern, ReadOnlyRecord, that will prevent a save when included in an ActiveRecord model.

```
ruby
module ReadOnlyRecord
  extend ActiveSupport::Concern

  included do
    afterinitialize :readonly!
```

```
  end
end
```

- Attempting to save (e.g. create, update, or destroy) one of these models would raise an error (e.g. ActiveRecord::ReadOnlyRecord: Subscription is marked as readonly)
- Even with this code, a record could still be deleted with record.delete. We could write a RuboCop rule for avoiding using delete (possibly even for just these ReadOnlyModels). We could also overwrite this method to raise an error as well.
- Within certain namespaces like the Zuora subscriptions local copy sync service, we want access to a model that have write privileges.

Rollout of Zuora subscriptions local copy

With the first iteration of introducing the models for Zuora subscriptions local copy, we will take an iterative approach to the rollout. There should be no impact to existing functionality as we build out the models, start populating the data through callouts, and backfill these models. Once this is in place, we will iteratively update existing features to use the Zuora subscriptions local copy instead of querying Zuora directly.

We will make this transition using many small scoped feature flags, rather than one large feature flag to gate all of the new logic using Zuora subscriptions local copy. This will help us deliver more quickly and reduce the length with which feature flag logic is maintained and test cases are retained.

Testing can be performed before Zuora subsctiptions local copy is used in the codebase to ensure data integrity of the models of subscriptions local copy.

Phase three: Utilize Zuora subscriptions local copy

This phase covers the third phase of work of the Orders re-architecture. In this phase, the focus will be utilizing the new models for Zuora subscriptions local copy introduced in phase one. Querying Zuora for Subscription data is fundamental to Customers so there are plenty of places that will need to be updated. In the places where CDot is reading from Zuora, it can be replaced by querying the local copy instead. This should result in a big performance boost by avoiding third party requests, particularly in components like the Seat Link Service.

This transition will be completed using many small scoped feature flags, rather than one large feature flag to gate all of the new logic using these models for local copy. This will help to deliver more quickly and reduce the length with which feature flag logic is maintained and test cases are retained.

Phase four: Transition from Order to Subscription

The fourth phase for this blueprint focuses on trimming the orders table and resolving data consistency issues.

1. Trimming orders table

We want to go over the below attributes and evaluate if their functionality can be replaced

with methods. If it is feasible, we should remove the column from the orders table and add a new method for it in Order model.

- billingaccountid
- productrateplanid
- subscriptionid
- startdate
- enddate
- quantity
- amendmenttype
- source

Current schema of orders table

Column	Action
customerid	Handle in trials data migration
productrateplanid	Evaluate and remove if feasible
subscriptionid	Evaluate and remove if feasible
subscriptionname	Keep
startdate	Evaluate and remove if feasible
enddate	Evaluate and remove if feasible
quantity	Evaluate and remove if feasible
glnamespaceid	Keep
glnamespacename	Keep
amendmenttype	Evaluate and remove if feasible
trial	Handle in trials data migration
lastextraciminutessyncat	Keep
zuoraaccountid	Keep
increasedbillingratenotifiedat	Keep
reconciliationaccepted	Keep
source	Evaluate and remove if feasible
seatoveragenotifiedat	Keep
autorenewerrornotifiedat	Keep
billingaccountid	Evaluate and remove if feasible
monthlyseatdigestnotifiedon	Keep
sourceglnamespaceid	Keep
trialtype	Handle in trials data migration

- Trials data migration is being done as part of https://gitlab.com/gitlab-org/customers-gitlab-com/-/issues/11047

2. Resolving data issues

There should be only one order per subscription name, but there are a few duplicates present. These duplicates are created because of the current behavior when processing an Order Processed callout in CDot if the zuoraaccountid changes for a Zuora Subscription.

1. The Billing Account Membership is updated to the new Billing Account for the CDot Customer matching the Sold To email address.

1. CDot attempts to find the CDot Order with the new billingaccountid and subscriptionname.
1. If an Order isn't found matching this criteria, a new Order is created. This leads to two Order records for the same Zuora Subscription.

This should be fixed and existing duplicates should be removed.

To resolve existing duplicates we need to -

1. Determine which record to keep in case of duplicate, and add rake task to delete the appropriate data
1. Add unique db constraint and model validation for subscriptionname + zuora account id

Subscription related data such as startdate, enddate, quantity, amendmenttype can be delegated to the latest subscription.

To find the latest subscription you just need its name:

- ID refers to a specific version
- Name is common for all versions of a subscription

The zuorasubscriptionid could be set to the latest version on typical updates. Most of the data on Order is GitLab metadata (e.g. lastextraciminutessyncat) so it wouldn't need to be updated.

3. Rename Order and/or Subscription (TBD)

Renaming the Order model and orders table could eliminate confusion around the Order model. The data stored in the CustomersDot Order model does not correspond to a Zuora Order. As Order more closely resembles a Zuora Subscription with some additional metadata about syncing with GitLab.com, it could be renamed to Subscription.

The rename of Order model is up for debate given a Subscription model already exists.

Proposed DB schema

```
mermaid
erDiagram
  Order ||--|{ "Zuora::Local::Subscription" : "has many"

  Order {
    bigint id PK
    string(64) zuoraaccountid
    string(64) zuorasubscriptionid
    string zuorasubscriptionname
    string gitlabnamespaceid
    string gitlabnamespacename
    datetime lastextraciminutessyncat
    datetime increasedbillingratenotifiedat
    boolean reconciliationaccepted "null:false default:false"
    datetime seatoveragenotifiedat
    datetime autorenewerrornotifiedat
```

```
  date monthlyseatdigestnotifiedon
  datetime createdat
  datetime updatedat
}

"Zuora::Local::Subscription" {
  string(64) zuoraid PK "id field on Zuora Subscription"
  string(64) accountid
  string name
}
```

Notes

- This model serves as a record of the Subscription that is modifiable by the CDot application, whereas the Zuora::Local::Subscription table below should be read-only.
- There will be one Subscription record per actual subscription instead of a Subscription version.
  - This has the advantage of avoiding duplication of fields like gitlabnamespaceid or lastextraciminutessyncat.
  - The zuorasubscriptionid column could be removed or kept as a reference to the latest Zuora Subscription version.

Trial data

The CDot Order model contains paid subscription data, as well as trials data such as customerid, trial, trialtype.

New tables and models for trial data were added in Build new trial structures. The migration of the trials data to these new structures is being handled as part of Move GitLab.com Trials to use new data structure.

Resources

- FY24Q3 OKR - Create plan to align CustomersDot Orders to Zuora Orders
- Epic &9748 - Align CustomersDot Orders to Zuora objects

SECTION: engineering/architecture/design-documents/cdot_plan_managment/_index.md

{{< engineering/design-document-header >}}

Summary

The GitLab Customers Portal is an independent application, distinct from the GitLab product, designed to empower GitLab customers in managing their accounts, subscriptions, and conducting tasks such as renewing and purchasing additional seats. More information about the Customers Portal can be found in the GitLab docs. Internally, the application is known as CustomersDot (also known as CDot).

GitLab uses Zuora's platform as the SSoT for all product-related information. The Zuora Product Catalog represents the full list of revenue-making products and services that are salable, or have been sold by GitLab, which is core knowledge for CustomersDot decision making. CustomersDot currently has a local copy of the Zuora Product Catalog and refreshes it daily through a scheduled job. However, every time a new Product, Product Rate Plan, or Product Rate Plan Charge is updated or added to the Zuora Product Catalog, additional manual effort is required to make it available in CustomersDot.

CustomersDot uses Plan as a wrapper class for easy access to all the details about a Plan in the Product Catalog. Given that the name, price, minimum quantity, and other details of the Plan are spread across the Zuora::ProductRatePlan, Zuora::ProductRatePlanCharge, and Zuora::ProductRatePlanChargeTier objects, traditional access to these details can be cumbersome. This class is very useful because it saves us from having to query for all these details. Additionally, the class helps with the classification of Zuora::ProductRatePlans based on their tier, deployment type, and other criteria used across the app.

The main goal of this design document is to improve the architecture and maintainability of the Plan model within CustomersDot. When the Product Catalog is updated in Zuora, it should automatically reflect in CustomersDot without requiring app restarts, code changes, or manual intervention.

Motivation

Every time a new Product/SKU is added to the Zuora Product Catalog, despite having a daily refreshed local copy, it requires code changes in CustomersDot to make it available. This is due to the current strategy the Plan class uses for classification, which consists of assigning the Zuora::ProductRatePlan IDs to constants and then manually forming groups of IDs to represent different categories like all plans in the Ultimate tier or all the add-ons available for self-procurement for GitLab.com. These categories are then used for decision-making during execution.

As the codebase and number of products grow, this manual intervention becomes more expensive.

Goals

Automate the Plan management in CustomersDot so it will require no manual intervention for basic Product Catalog updates in Zuora. For example, when a new Product/SKU is added, a RatePlanCharge is updated, or a Product is discontinued. To achieve this, we need to move away from statically defining product rate plan IDs within CustomersDot and transfer the classification knowledge to the Zuora Product Catalog (by adding CustomersDot metadata to it in the form of custom fields) to be able to resolve these sets dynamically.

Decisions

1. ADR-001 Use JSONB for Classification Metadata Storage

Proposal

Transfer CustomersDot's classification knowledge to the Zuora Product Catalog to resolve ProductRatePlans by querying our Product Catalog local copy dynamically. This transfer can be

handled in iteration following the flow represented below until all the plan constants that refer to ProductRatePlan IDs are replaced and removed.

mermaid
sequenceDiagram
    autonumber
    participant FTE as Fulfillment Team Engineer
    participant EntApps as EntApps Team
    participant CDot as CustomersDot
    participant ZuoraAPI as Zuora API
    participant ZuoraDB as Zuora Database
    participant LocalDB as Local DB Copy

    Note over FTE, LocalDB: One-time setup phase
    FTE->>CDot: Create migration to add JSONB column
    CDot->>LocalDB: Apply migration to add customfields JSONB column
    Note over LocalDB: JSONB column ready to store all custom fields

    Note over FTE, LocalDB: Iterative process for each field/set of fields
    FTE->>EntApps: Submit Change Request issue to create custom fields in Zuora
    EntApps->>ZuoraDB: Create custom fields (e.g., cdotpurchasablec)
    Note over ZuoraDB: Custom fields added to ProductRatePlan table

    FTE->>CDot: Develop script to extract classification knowledge from Plan class
    CDot->>ZuoraAPI: Update ProductRatePlans with values based on Plan constants
    ZuoraAPI->>ZuoraDB: Save custom field values
    Note over ZuoraDB: ProductRatePlan records populated with classification data

    Note over CDot: Later - during scheduled sync
    CDot->>ZuoraAPI: Request ProductCatalog (including new custom fields)
    ZuoraAPI->>CDot: Return ProductCatalog with custom field values
    CDot->>LocalDB: Refresh local copy, storing field values in customfields
    Note over LocalDB: JSONB column now contains key-value pairs for custom fields

    CDot->>CDot: CDot logic can now use these fields from local copy
    Note over CDot: Replace Plan constants with queries on customfields

New Custom Fields

mermaid

erDiagram
  "Product" ||--|{ "ProductRatePlan" : "has many"
  "ProductRatePlan" ||--|{ "ProductRatePlanCharge" : "has many"
  "ProductRatePlanCharge" ||--|{ "ProductRatePlanChargeTier" : "has many"

  "ProductRatePlan" {
    jsonb customfields "Local storage for all c fields"
    %% Fields stored within customfields JSONB:

```
  boolean cdotaccessiblec "· in customfields"
  array cdotactionsc "· in customfields"
  enum cdotplanstatusc "· in customfields"
  boolean cdotistrueupc "· in customfields"
  boolean cdotisuspubsecc "· in customfields"
  enum cdotcommunitytypec "· in customfields"
 }

 "ProductRatePlanCharge" {
  enum billingperiod "Zuora field"
  jsonb customfields "Local storage for all c fields"
  %% Fields stored within customfields JSONB:
  enum cdotplantypec "· in customfields"
  enum chargetierc "· in customfields"
  enum chargedeploymentc "· in customfields"
 }
```

Field Name	Level	New Field?	Data Type	Values	Description
CDotAccessiblec	ProductRatePlan	Yes	Boolean	true, false	Indicates whether a plan is accessible within CustomersDot. Plans marked true are displayed to users and their details can be viewed, regardless of purchase origin. Plans marked false exist in Zuora but are completely invisible in CustomersDot.
CDotActionsc	ProductRatePlan	Yes	Multiselect	initialpurchase, additionalpurchase, renew	Specifies the actions available for this plan within CustomersDot. Multiple actions can be selected: · initialpurchase: Plan can be purchased self-service through the Customers Portal without sales assistance. · additionalpurchase: Additional quantity of this plan can be purchased self-service through the Customers Portal. · renew: Subscription with this plan can be renewed self-service through the Customers Portal · No option selected is a valid state and results in these actions not being available in the Customers Portal.
CDotPlanStatusc	ProductRatePlan	Yes	String	active, deprecated, legacy, notapplicable	Represents the lifecycle stage of a plan: · active: Currently salable and fully supported / available plans · deprecated: Plans being phased out but still available to existing customers · legacy: Historical plans maintained only for existing subscriptions · notapplicable: Special cases where status concept doesn't apply
CDotIsTrueUpc	ProductRatePlan	Yes	Boolean	true, false	Identifies true-up plans, which are special product rate plans used to reconcile usage beyond what was initially purchased.
CDotIsUsPubSecc	ProductRatePlan	Yes	Boolean	true, false	Identifies plans specifically designed for US Public Sector customers.
CDotCommunityTypec	ProductRatePlan	Yes	String	education, opensource, startup, notapplicable	Identifies special pricing programs for specific communities: · education: Educational institutions · opensource: Open source projects · startup: Startup companies · notapplicable: Standard commercial plans
CDotPlanTypec	ProductRatePlanCharge	Yes	String	ciminutes, storage, duopro, duoenterprise, duoamazonq, agileplanning, productanalytics, professionalservices, ecosystem, baseproduct, notapplicable	Categorizes charges by the services they provide: · ciminutes: Additional CI/CD pipeline minutes · storage: Additional repository storage · duopro: GitLab Duo Pro AI capabilities · duoenterprise: GitLab Duo Enterprise AI capabilities ·

duoamazonq: Amazon Q integration
· agileplanning: Enterprise Agile Planning features
· productanalytics: Product analytics capabilities
· professionalservices: Training, consulting, and implementation services
· baseproduct: Standalone charge e.g. Ultimate or Premium
· ecosystem: GitLab Ecosystem offering discount charge
· notapplicable: None of the mentioned |
BillingPeriod	ProductRatePlanCharge	No	String	monthly, annual, twoyear, threeyear, fouryear, fiveyear (or 1, 12, 24, 36, 48, 60)	Defines the duration of the billing cycle for the plan. Can use either named periods or the number of months.
ChargeTierc	ProductRatePlanCharge	No	String	Ultimate, Premium, Bronze, Legacy, Starter, Not Applicable, null	Represents the feature tier of a plan, with different tiers offering progressively more features: · ultimate: Most comprehensive feature set · premium: Advanced features · bronze/silver/gold: Legacy tier names · starter: Entry-level paid tier · free: No-cost tier with limited features
ChargeDeploymentc	ProductRatePlanCharge	No	String	Self-Managed, Dedicated, GitLab.com, Not Applicable, null	Indicates how the GitLab instance is deployed and managed: · Self-Managed: Customer installs and manages GitLab on their infrastructure · Dedicated: GitLab-managed single-tenant instance · GitLab.com: Multi-tenant SaaS offering at gitlab.com

Additional Considerations

- Field names match Zuora custom field naming conventions with the c suffix
- New fields added specifically for CDot are prefixed with CDot

Design and implementation details

Our classification is at the ProductRatePlan, ProductRatePlanCharge levels. As a first step we will add a column (JSONB) to our local copy of ProductRatePlan and ProductRatePlanCharge to persist this classification.

ruby
example migration
```
class AddCustomFieldsToProductRatePlans < ActiveRecord::Migration[7.1]
  def change
    addcolumn :zuoraproductrateplans, :customfields, :jsonb, default: {}, null: false,
      comment: columncomment
    addindex :zuoraproductrateplans, :customfields, using: :gin
  end

  private

  def columncomment
    {
      owner: 'section::fulfillment',
      dataclassification: 'orange',
      description: 'Stores plan classification metadata as key-value pairs.'
    }.tojson
  end
end
```

```
class AddCustomFieldsToProductRatePlanCharges < ActiveRecord::Migration[7.1]
  def change
    addcolumn :zuoraproductrateplancharges, :customfields, :jsonb, default: {}, null: false,
      comment: columncomment
    addindex :zuoraproductrateplancharges, :customfields, using: :gin
  end

  private

  def columncomment
    {
      owner: 'section::fulfillment',
      dataclassification: 'orange',
      description: 'Stores plan charge classification metadata as key-value pairs.'
    }.tojson
  end
end
```

Iteration Plan

We will iterate over the proposed custom fields picking one field / set of fields at a time and:

1. Submit a Change Request to EntApps to add the necessary field(s) to Zuora.
2. Transfer the CustomersDot knowledge to the Zuora Product Catalog by populating the new field(s) and keeping them in sync during rollout.
3. Confirm that the Product Catalog copy has synced correctly (either manually trigger the sync or wait for the scheduled daily sync).
4. [Behind a feature flag] Replace any usage of Plan constants that represent a collection of records that meet a given classification with a call to a method that loads the same collection from the local copy of the Product Catalog leveraging the custom field.
5. Validate both logic and performance in the staging environment.
6. Deploy the change to production and enable it for all users.

The following code example illustrates steps 4 from the iteration process described above. It shows how we would replace hardcoded constants in the Plan class with dynamic methods that leverage the custom fields from our local Product Catalog copy. This example specifically demonstrates migrating from hardcoded constants for SaaS plans to dynamic queries based on the cdotactionsc and chargedeploymentc fields.

ruby
app/models/zuora/local/productrateplancharge.rb

```
customfield :deployment, remotename: :chargedeploymentc, type: :string

scope :gitlabcom, -> { jsonbcontains(deployment: 'GitLab.com') }
```

app/models/zuora/local/productrateplan.rb

```
customfield :actions, remotename: :cdotactionsc, type: :zuoramultiselectselection
```

```
scope :cdotpurchasable, -> { jsonbcontains(actions: 'initialpurchase') }
scope :gitlabcom, lambda {
        joins(:productrateplancharges)
        .merge(Zuora::Local::ProductRatePlanCharge.gitlabcom)
        .distinct
      }

 lib/planclassifier.rb
module PlanClassifier
  def self.allgitlabcomplans
    Zuora::Local::ProductRatePlan.gitlabcom.pluck(:zuoraid)
  end

  def self.selfservicegitlabcomplans
    Zuora::Local::ProductRatePlan.cdotpurchasable.gitlabcom.pluck(:zuoraid)
  end
end

 In app/models/plan.rb
class Plan
   before
  def self.selfservicegitlabcomplans
    @@selfservicegitlabcomplans ||= ALLSELFSERVICESAASPLANS
  end

   after
  def self.selfservicegitlabcomplans
    PlanClassifier.selfservicegitlabcomplans
  end

   before
  def self.allgitlabcomplans
    @@allgitlabcomplans ||= [
      BASICSAAS1YEARPLAN,
      PREMIUMSAASPLANS,
      ULTIMATESAASPLANS,
      GITLABCOMBRONZEPLANS,
      DEPRECATEDSILVERSAASPLANS,
      DEPRECATEDGOLDSAASPLANS,
      ALLGITLABCOMEDUOSSPLANS,
      TRIALSAASPLANS
    ].flatten.compact
  end

   after
  def self.allgitlabcomplans
    PlanClassifier.allgitlabcomplans
  end
```

As a first iteration we can replace the True up related logic as it doesn't have provision implications.

Validation Strategy

1. Data Integrity: Compare classifications from both approaches using a rake task
2. Performance: Benchmark queries using both approaches
3. Coverage: Ensure all existing use cases are covered by metadata-based approach

Rollback Plan

Each feature flag provides a built-in rollback mechanism. If issues are detected:

1. Disable the feature flag
2. Return to using the constant-based approach
3. Document issues for resolution
4. Re-enable once fixed

Implementation Timeline

1. Phase 1 (FY2026Q1):
   1. Implement JSONB column and basic classification fields
   2. Create Change Requests (EntApps) to add the custom fields to the ProductRatePlan in Zuora
   3. Update CustomersDot syncing mechanism so it can sync metadata dynamically
   4. Refactor codebase so the Plan constants are used within Plan class ONLY and external usages are through methods so we can replace each method's approach once the metadata is available.
   5. Establish baseline metrics for plan-related operations

2. Phase 2 (FY2026Q2) In iteration, starting with high-priority constants:
   1. Transfer CustomersDot knowledge by using a rake task to populate the custom fields added in the previous iteration
   2. Validate that metadata queries return identical results to constant-based approach
   3. Replace constants with metadata queries
   4. Compare performance metrics after implementation
   5. Update documentation, record demos and conduct knowledge sharing sessions (specially after the first iteration so new additions can to Plan can follow it)
   6. Rollout

Priority criteria: "high-priority" constants will be determined by usage frequency.

Risk Mitigation

Risk	Mitigation
Incomplete custom field population in Zuora	Implement validation checks in the synchronization process
Performance impact of JSONB queries	Add monitoring and benchmarking; create indexes on common query patterns
Discrepancies between old and new classification	Create reconciliation reports to compare

classifications |
| Edge cases not covered by metadata | Document process for handling special cases |

SECTION: engineering/architecture/design-documents/cdot_plan_managment/decisions/001_jsonb_for_classification_metadata.md

Context

We needed a flexible storage solution for plan classification metadata that would allow us to add new fields without requiring database migrations for each addition.

Decision

We will use a JSONB column named customfields on the zuoraproductrateplans table to store all classification metadata.

Alternatives Considered

1. Plain Columns
   - · Strong schema validation and better query performance
   - · Requires migrations for each new field, less flexible, more complex to maintain

2. HSTORE
   - · Optimized for flat key-value structures with potentially better performance
   - · No need for migrations when adding new metadata fields
   - · Only supports string values, requiring type conversion
   - · Requires enabling a PostgreSQL extension

3. JSONB (chosen)
   - · Preserves native data types (booleans, numbers)
   - · No extension required (built into PostgreSQL)
   - · Team already familiar with JSONB
   - · Provides flexibility for future needs
   - · Potentially slightly slower than HSTORE for key-value lookups (not significant for our scale)

Rationale

With our dataset being relatively small (~800 records) and our query patterns focused on simple filtering through scopes without complex ordering, the performance advantages of HSTORE were outweighed by JSONB's broader benefits. The familiarity of the team with JSONB, its native data type preservation, and not requiring a database extension made it the most practical choice.

SECTION: engineering/architecture/design-documents/cells/_index.md

{{< engineering/design-document-header >}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future.

Cells is a new architecture for our software as a service platform. This architecture is horizontally scalable, resilient, and provides a more consistent user experience. It may also provide additional features in the future, such as data residency control (regions) and federated features.

For more information about Cells, see also:

Cells Iterations

- The Cells 1.0 target is to deliver a solution
  for internal customers using the SaaS GitLab.com offering, and foundational work for Cells.
- The Cells 1.5 target is to deliver a migration solution
  for existing and new enterprise customers using the SaaS GitLab.com offering, built on top of the Cells 1.0 architecture.
- The Cells 2.0 target is to support a public and open source contribution
  model in a cellular architecture.

Goals

See Goals, Glossary and Requirements.

Technical proposals

The Cells architecture has long lasting implications to data processing, location, scalability and the GitLab architecture.
This section links all different technical proposals that are being evaluated.

- Cells Services:
  - HTTP Routing Service
  - SSH Routing Service
  - Topology Service
  - Planned: Indexing Service
- Mutual authentication between Cell services
- Feature Flags - (Previous iteration)
- Cells: Infrastructure
- Organization migration
- Routable Tokens

Impacted features

The Cells architecture will impact many features requiring some of them to be rewritten, or changed significantly.
Below is a list of known affected features with preliminary proposed solutions.

- Cells: Admin Area
- Cells: Advanced search
- Cells: Backups

- Cells: CI/CD Catalog
- Cells: CI Runners
- Cells: Container Registry
- Cells: Contributions: Forks
- Cells: Data Migration
- Cells: Explore
- Cells: Git Access
- Cells: Global Search
- Cells: GraphQL
- Cells: Organizations
- Cells: Personal Access Tokens
- Cells: Personal Namespaces
- Cells: Secrets & Credentials
- Cells: Snippets
- Cells: User Profile
- Cells: Your Work

Impacted features: Placeholders

The following list of impacted features only represents placeholders that still require work to estimate the impact of Cells and develop solution proposals.

- Cells: Agent for Kubernetes
- Cells: Data pipeline ingestion
- Cells: GitLab Pages
- Cells: Group Transfer
- Cells: Issues
- Cells: Merge Requests
- Cells: Project Transfer
- Cells: Router Endpoints Classification
- Cells: Schema changes (Postgres and Elasticsearch migrations)
- Cells: Uploads
- ...

Frequently Asked Questions

What's the difference between Cells architecture and GitLab Dedicated?

We've captured individual thoughts and differences between Cells and Dedicated over here

The new Cells architecture is meant to scale GitLab.com.
The way to achieve this is by moving Organizations into Cells, but different Organizations can still share server resources, even if the application provides isolation from other Organizations.
But all of them still operate under the existing GitLab SaaS domain name gitlab.com.
Also, Cells still share some common data, like users, and routing information of Groups and Projects.
For example, no two users can have the same username even if they belong to different Organizations that exist on different Cells.

With the aforementioned differences, GitLab Dedicated is still offered at higher costs due to the fact that it's provisioned with dedicated server resources for each customer, while Cells use shared resources.
This makes GitLab Dedicated more suited for bigger customers, and GitLab Cells more suitable for small to mid-size companies that are starting on GitLab.com.

On the other hand, GitLab Dedicated is meant to provide a completely isolated GitLab instance for any Organization.
This instance is running on its own custom domain name, and is totally isolated from any other GitLab instance, including GitLab SaaS.
For example, users on GitLab Dedicated don't have to have a different and unique username that was already taken on GitLab.com.

Can different Cells communicate with each other?

Not directly, our goal is to keep them isolated and only communicate using global services.

How are Cells provisioned?

The GitLab.com cluster of Cells uses GitLab Dedicated tooling for provisioning GitLab Instances.
That's why Cells are referred to by Tenants in some projects.
Once any Cell instance gets provisioned it could join the GitLab.com cluster and become a Cell.
One requirement will be that the instance does not contain any prior data. One of the reasons is that Cells save data with custom primary key ranges that they pull from the Topology Service.

The Cells are managed in the tissue
project, where we manage all the Cells for both dev and prod environments
in the rings directory.

Each cells configuration, is validated against tenant-model-schema
which is already used for GitLab dedicated tenants as well.

Deployment Process

The deployment workflow follows these steps:

1. The instrumentor retrieves the Cell configuration, which in Dedicated Tooling known as TENANTMODEL.
2. Instrumentor parses the TENANTMODEL and pass the required configuration to GET (GitLab Environment Toolkit).
3. GET deploys the infrastructure and uses Helm Installation to install GitLab in the provisioned Kubernetes Cluster.

This approach aligns with how we deploy GitLab on the existing legacy Cell infrastructure in both Staging and Production environments through Kubernetes workloads.

>[!note]
>This is a high-level overview. For more detailed information, refer to the Dedicated Architecture Documentation.

To reach shared resources, Cells will use Private Service Connect.

See also the design discussion.

What is a Cells topology?

See the design discussion.

How are users of an Organization routed to the correct Cell?

TBD

How do users authenticate with Cells and Organizations?

See the design discussion.

How are Cells rebalanced?

TBD

How can Cells implement disaster recovery capabilities?

TBD

How do I decide whether to move my feature to the cluster, Cell or Organization level?

By default, features are required to be scoped to the Organization level. Any deviation from that rule should be validated and approved by Tenant Scale.

The design goals of the Cells architecture describe that all Cells are under a single domain and as such, Cells are invisible to the user:

- Cell-local features should be limited to those related to managing the Cell, but never be a feature where the Cell semantic is exposed to the customer.
- The Cells architecture wants to freely control the distribution of Organization and customer data across Cells without impacting users when data is migrated.

Cluster-wide features are strongly discouraged because:

-